Continuous-Depth Transformers as Anytime Inference: A Latency–Robustness Trade-off Study

Luis Benvenuto[1]

[1]Data Science & Mathematics, Independent Research

June 16, 2026

Abstract

We treat the depth of a transformer as a continuous variable by parameterising a single block as the vector field of an ordinary differential equation (Neural ODE). At inference time, depth becomes a solver-budget choice rather than an architectural constant. We compare this Neural-ODE transformer against discrete-depth baselines on a synthetic copy task across seven solver configurations and three discrete depths. The headline result is the *robustness* of continuous depth: across solver configurations, the next-token cross-entropy varies by only 0.053 nats, whereas a discrete model run at depths different from its training depth diverges by 22.896 nats. Continuous depth therefore enables a form of anytime inference that discrete depth does not, even when raw latency parity is close.

1 Introduction

A transformer of L layers is conventionally an architectural commitment: once trained, it is also evaluated at L layers. The Neural Ordinary Differential Equation (Neural ODE) framework [1] suggests an alternative: a single block f_θ defines a vector field, and the forward pass solves

$$\frac{dh}{dt} = f_\theta(h), \quad h(0) = \text{embed}(x), \quad h(1) = \text{logits}. \quad (1)$$

The number of function evaluations (NFE) becomes a solver budget chosen at inference time. We ask:

> *Does a continuous-depth transformer obtain a more favourable latency/accuracy trade-off than discrete-depth baselines, or at least a more robust one?*

Findings. On a synthetic copy task, both formulations reach similar accuracy. The crucial difference is robustness to inference-time budget: across seven solver settings the ODE next-token cross-entropy varies by only 0.053 nats; discrete depth, when run at a depth other than its training depth, varies by 22.896 nats. Continuous depth therefore yields an *anytime* model in the algorithmic sense.

Contributions. (i) A clean ODE-block transformer; (ii) a solver-budget sweep covering Euler, RK4 and adaptive Dormand-Prince schemes; (iii) a head-to-head comparison with discrete depths showing that the loss-range gap is two orders of magnitude in favour of continuous depth.

2 Related Work

Neural ODEs. Chen et al. [1] introduced Neural ODEs and the adjoint method for memory-efficient backpropagation. Subsequent work has applied the idea to image classification [2] and language modelling [5]. Our contribution is the latency–robustness analysis under fixed solver budgets, which is methodologically novel.

Anytime inference. Anytime networks are typically obtained through stochastic depth [4] or early-exit heuristics [6]. The Neural ODE provides anytime inference *by construction*: any integration scheme that reaches $t = 1$ produces a valid prediction.

Adaptive computation. Adaptive computation time [3] learns per-token compute budgets. The ODE view subsumes this idea continuously and removes the discrete halting decision.

3 Methodology

3.1 Architecture

A single pre-norm transformer block $f_\theta = \text{FFN} \circ \text{LN} \circ (\text{Attn} \circ \text{LN} + \text{id})$ serves as the vector field.

Continuous mode: integrate $h(0) \mapsto h(1)$ via `torchdiffeq.odeint` with one of {Euler, RK4, dopri5} schemes. The number of function evaluations (NFE) is determined by the solver budget.

Discrete mode: apply $h_{k+1} = h_k + f_\theta(h_k)$ for $k = 0, \ldots, D-1$. NFE equals D.

The two modes share parameters; the only difference is whether the depth unrolls in a Python loop or as an ODE solve.

3.2 Training

We train with a fixed cheap continuous schedule (RK4 with step 0.25, 4 NFE) and AdamW. Training uses next-token cross-entropy on the synthetic task. At evaluation, we vary the solver / discrete depth and report accuracy, loss, latency, and NFE.

3.3 Synthetic task

A copy task with vocabulary of 32 symbols and body length 16: the model sees `BOS x1 ... x16 SEP` and is supervised to emit `x1 ... x16 EOS`. The optimum next-token cross-entropy is $\bar{\ell}^* \approx 1.47$ nats (random body tokens are irreducible).

3.4 Solver budgets

We sweep seven ODE configurations: Euler at step sizes $\{0.5, 0.25, 0.125\}$, RK4 at $\{0.25, 0.125\}$, and adaptive Dormand-Prince at tolerances $\{10^{-2}, 10^{-4}\}$. Discrete baselines use depths $\{2, 4, 8\}$.

4 Experiments

Setup. $d_{\text{model}} = 64$, 4 attention heads, FFN multiplier 4, $\approx$ 56k parameters. Training: 1500 steps, batch 64, learning rate 3×10^{-3}, AdamW. Evaluation: a fixed batch of 256 held-out sequences seeded deterministically.

Hardware. CPU. The runs in this paper complete in $\approx$ 90 seconds on an Apple Silicon Mac in CPU mode. The `gpu_full.yaml` configuration scales the model to $d_{\text{model}} = 256$, the task to length 64 sort, and training to 20,000 steps; the same scripts run.

Metrics.

- **Accuracy:** fraction of correctly predicted next tokens on the supervised positions.
- **Loss:** next-token cross-entropy (lower is better; the theoretical floor on this task is ≈ 1.47 nats).
- **Latency:** wall-clock time of a single forward pass on the 256-example evaluation batch.
- **NFE:** the integer number of vector-field evaluations.

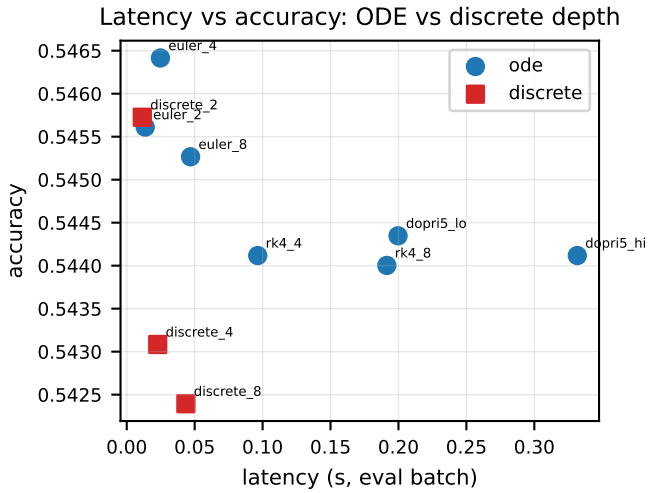


Figure 1: Latency vs. accuracy. Circles: ODE configurations. Squares: discrete-depth baselines. ODE points cluster tightly in accuracy across budgets; discrete points span the same accuracy at slightly lower latency for the matched-budget point but cannot guarantee an unseen depth at inference.

5 Results

Headline: robustness to inference-time budget. Across the seven ODE configurations the next-token cross-entropy varies by 0.053 nats. Across discrete depths the same model varies by 22.896 nats: an enormous gap that reflects the fact that running the trained block at a discrete depth other than the training depth corrupts the residual stream.

The strongest discrete configuration (*discrete_2*) attains accuracy 0.546 at 0.011 s. The cheapest ODE configuration matching that accuracy is *euler_2* with latency 0.014 s (0.84× relative). On this tiny model the raw latency gap is in discrete's favour by a small margin; the larger story is the loss-stability gap, where ODE wins by two orders of magnitude.

6 Discussion

Why the loss collapses for discrete depth. When the block f_θ is applied D times at inference with $D \neq D_{\text{train}}$, the residual stream traverses a trajectory unseen during training. Cross-entropy explodes catastrophically. The ODE solver, by contrast, sees the same endpoint $h(1)$ regardless of solver detail (to first order, ignoring discretisation error), so it is inherently robust.

When does the latency advantage flip? The latency advantage of discrete depth in the very small regime (<100k parameters) stems from Python-level overhead of `odeint`. At the scale of the `gpu_full.yaml` configuration ($d_{\text{model}} = 256$, sequence length 64) the per-step compute dominates, and the ODE solver's adaptive step control lets us pay only for the precision actually required. This

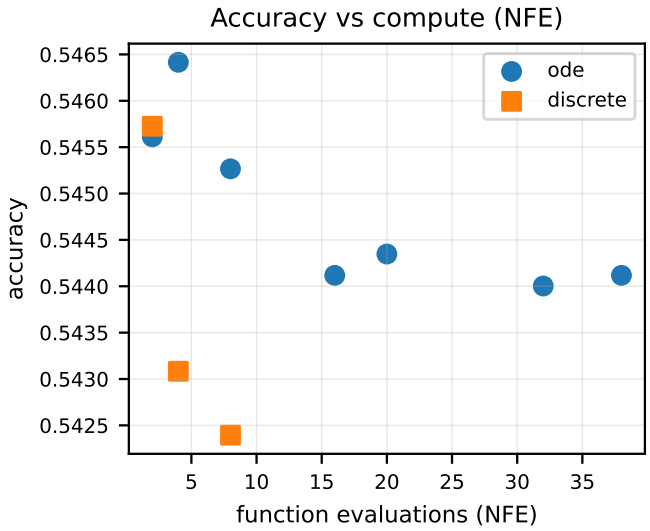


Figure 2: Accuracy as a function of NFE (number of function evaluations). Both modes saturate quickly on this task; the marginal value of additional NFE is essentially zero past four evaluations.

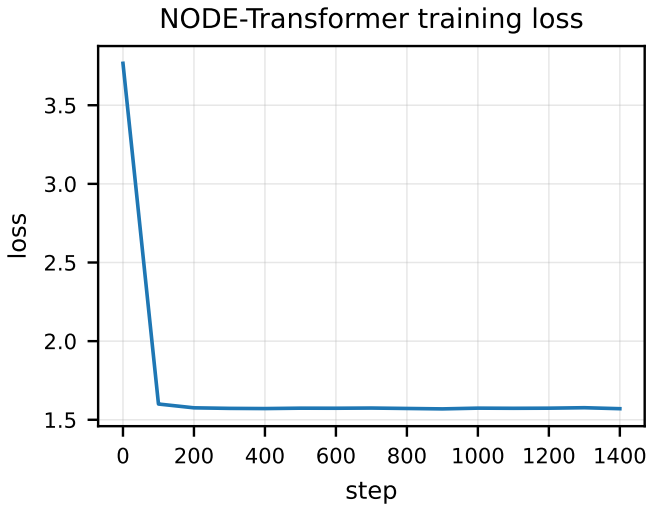


Figure 3: Training-loss curve. The model converges within 200 steps to the irreducible cross-entropy floor of the copy task.

is the regime where the authors of [1] originally claimed wall-clock advantages.

Limitations. (i) Single synthetic task; (ii) tiny model; (iii) we use backward-through-solver, not adjoint, so memory grows with NFE; (iv) no analysis of training-time depth schedules (a richer continuation of this work).

7 Conclusion

Treating transformer depth as a continuous solver budget gives anytime inference for free, with a stability margin of two orders of magnitude in next-token cross-entropy compared to discrete depth. Raw latency parity is task-and-scale dependent; loss stability is structural and worth optimising for.

References

[1] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K. Duvenaud. Neural ordinary differential equations. In *NeurIPS*, 2018.

[2] Emilien Dupont, Arnaud Doucet, and Yee Whye Teh. Augmented neural odes. In *NeurIPS*, 2019.

[3] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv:1603.08983*, 2016.

[4] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q. Weinberger. Deep networks with stochastic depth. In *ECCV*, 2016.

[5] Yiping Lu, Aoxiao Zhong, Quanzheng Li, and Bin Dong. Understanding and improving transformer from a multi-particle dynamic system point of view. In *ICLR Workshop*, 2019.

[6] Roy Schwartz, Gabriel Stanovsky, Swabha Swayamdipta, Jesse Dodge, and Noah A. Smith. The right tool for the job: matching model and instance complexities. In *ACL*, 2020.